8

Adding Media Playback to Packtflix with ExoPlayer

In the journey of Android development, the ability to create rich, engaging multimedia applications is a crucial skill that sets apart great apps from the good ones. As we venture further into the creation of our Netflix-like app, we'll transition from the foundational structures and user interfaces for browsing movie lists and details to the core of multimedia experiences: video playback. This chapter is dedicated to unlocking the potential of video content within our application, a feature that will significantly enhance user engagement and satisfaction. Here, we will travel into the world of media playback on Android, focusing on the powerful and versatile library known as ExoPlayer.

ExoPlayer stands out in the Android ecosystem as a robust, open-source library that provides an alternative to the standard Android MediaPlayer API. It offers extensive customization options and supports a wide range of media formats, including those not natively supported by Android. Our exploration will begin with an overview of media options in Android, setting the stage for why ExoPlayer is the library of choice for modern Android applications seeking to offer a superior media playback experience.

Following the introduction to media options, we will learn the basics of ExoPlayer, covering its architecture, key components, and how it integrates within an Android application. This foundational knowledge will prepare us to tackle the practical aspects of implementing video playback. This chapter will guide you through creating a responsive, intuitive video playback UI that meets the expectations of today's users.

The journey will continue with hands-on examples and detailed guidance on playing videos using ExoPlayer. This includes managing playback controls, adjusting video quality, and handling various media sources. Additionally, while recognizing the importance of accessibility and global reach, you'll learn how to add subtitles to your videos, ensuring your content is accessible to a wider audience.

By the end of this chapter, you will have mastered the essentials of video playback in Android, equipped with the skills to enrich your applications with high-quality video content, creating immersive experiences for your users.

In this chapter, we will cover the following topics:

- Reviewing media options in Android
- Reviewing Android's media options
- Creating the video playback user interface
- Playing video using ExoPlayer
- Adding subtitles to the video player

Technical requirements

As in the previous chapter, you will need to have Android Studio (or another editor of your preference) installed.

We will continue working on the same project we started in ***Chapter 7***. You can find the complete code that we are going to build throughout this chapter in this book's GitHub repository: **https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-8**.

Reviewing Android's media options

Android, as a versatile mobile operating system, offers comprehensive support for various types of media, including but not limited to audio files (such as MP3, WAV, and OGG) and video content (such as MP4, WebM, and MKV). This broad support empowers developers to incorporate a wide range of media types into their applications that can be used for diverse user preferences and use cases. From educational apps that leverage video tutorials for learning to entertainment platforms streaming movies and music, media playback is at the heart of modern mobile applications, driving user engagement and satisfaction.

To start our journey, let's look at which options we have in the Android ecosystem so that we can choose the most appropriate option to build the playback functionality of our app. We will start with MediaPlayer API and VideoView before considering ExoPlayer.

Learning about the MediaPlayer API

The **MediaPlayer** API is a powerful and flexible class that allows Android developers to handle audio and video playback with a high degree of control. The API is designed to be easy to use yet capable of catering to complex media playback requirements.

Its main features are as follows:

- **Versatile media source support**: MediaPlayer can play media from various sources, including local files (such as device storage or SD cards), raw resources (which are bundled within the app), and network streams (HTTP/HTTPS).
- **Playback control**: It offers comprehensive control over media playback, including play, pause, stop, rewind, and fast-forward options, as well as the ability to seek specific timestamps.

- **Volume control**: The MediaPlayer API in Android allows developers to programmatically adjust the volume of audio playback. This is achieved through methods such as **`setVolume(float leftVolume, float rightVolume)`**, which controls the volume level of the left and right speakers independently. This feature is essential for creating applications that can dynamically adjust the playback volume based on specific user settings, environmental conditions, or application scenarios. For instance, an app might automatically lower the volume during nighttime hours or increase it in a noisy environment to enhance user experience.
- **Event handling**: MediaPlayer provides listeners that can be used to respond to media life cycle events, such as completion, preparation, error handling, and buffering updates.
- **Audio focus management**: Essential for apps that play audio, MediaPlayer can handle audio focus to ensure a smooth user experience when multiple apps potentially play sounds simultaneously.

As we can see, MediaPlayer provides the basic functionality we need for simple audio and video handling, so it could be a good solution for the following cases:

- **Music players**: MediaPlayer is well-suited for apps that play music or podcast files, whether it's stored locally or streamed over the internet
- **Video players**: Although MediaPlayer requires more setup for video playback compared to VideoView, it's ideal for custom video player applications where developers need control over rendering and playback
- **Game sound effects**: For games that need to play short sound effects, MediaPlayer can be used for its simplicity and ability to handle various audio formats

Here's an example of how to reproduce an audio file using MediaPlayer:

```
@Composable
fun AudioPlayerComposable() {
    val context = LocalContext.current
    val mediaPlayer = remember { MediaPlayer.create(
        context, R.raw.my_audio_file) }
    // Observe lifecycle to release MediaPlayer
    ObserveLifecycle(owner = ProcessLifecycleOwner.get()) {
        onExit = {
            mediaPlayer.release()
        }
    }
    Column(modifier = Modifier.padding(16.dp)) {
        Button(onClick = {
            if (!mediaPlayer.isPlaying) {
                mediaPlayer.start()
            }
        }) {
            Text("Play")
        }
        Button(onClick = {
            if (mediaPlayer.isPlaying) {
```

```
                mediaPlayer.pause() // Use pause or stop
                                    based on your need
            }
        }) {
            Text("Stop")
        }
    }
}
@Composable
fun ObserveLifecycle(owner: LifecycleOwner, onExit: () ->
Unit) {
    // Use DisposableEffect to manage lifecycle
    DisposableEffect(owner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_DESTROY) {
                onExit()
            }
        }
        owner.lifecycle.addObserver(observer)
        onDispose {
            owner.lifecycle.removeObserver(observer)
        }
    }
}
```

In this example, **`MediaPlayer.create()`** is used within the **`remember`** block to ensure that the media player is only instantiated once, maintaining this instance across recompositions of the composable. Then, the **`ObserveLifecycle`** composable function is used to observe the life cycle of the entire application (using **`ProcessLifecycleOwner`** here for simplicity). This function ensures that **`mediaPlayer.release()`** is called to free up resources when the app is destroyed, although you might adapt this to more specific life cycle events as needed.

The UI consists of two buttons for play and stop functionalities. The play button's **`onClick`** logic checks if the media is not currently playing before starting playback. This is done to avoid restarting the audio and video if the button is pressed during playback. Similarly, the stop button pauses the playback.

This example demonstrates how to integrate MediaPlayer with Jetpack Compose while managing the media player life cycle and providing a simple UI for controlling playback. You can find more examples in the official documentation: **https://developer.android.com/media/platform/mediaplayer**.

Although our example illustrates how to provide the playback control UI, we still need to show the video so that our users can watch it. This is where VideoView comes in.

Learning about VideoView

VideoView is a higher-level UI component in Android that encapsulates the functionality of MediaPlayer and SurfaceView to provide a convenient way to play video files. It simplifies the process of video playback by managing the underlying media playback mechanics, making it ideal for

use cases that require straightforward video playback without the need for fine-grained control over the media pipeline.

NOTE

SurfaceView is a specialized component in the Android framework that provides a dedicated drawing surface within the app's view hierarchy. Unlike standard views, which are drawn onto a single canvas managed by the UI thread, SurfaceView can be rendered independently in a separate thread. This allows for more efficient redrawing, especially for demanding content such as video playback or dynamic graphics. SurfaceView is particularly useful when you need to update your views frequently or when the rendering process is computationally intensive as it does not block user interaction while drawing.

Let's explore some of VideoView's features so that we can appreciate the practical benefits it offers:

- **Simplicity**: VideoView simplifies the implementation of video playback. You can start playing a video with just a few lines of code, handling preparation and playback of the video file automatically.
- **Control integration**: It can be easily integrated with media controls (using MediaController), allowing users to play, pause, and seek through the video.
- **Format support**: VideoView supports various video formats that Android's MediaPlayer supports, including MP4, 3GP, and more, depending on the device and platform version.
- **Layout flexibility**: Being a view, VideoView can be placed anywhere in your application's layout and can be resized and styled as needed, just like any other UI component.

Understanding VideoView's features sets the stage for its practical applications. Now, let's pinpoint exactly where VideoView shines. Here are the best scenarios for using VideoView in your app:

- **Simple video playback**: When you need to play videos without requiring advanced playback features such as adaptive streaming, VideoView is a straightforward and effective choice. Adaptive streaming, such as **HTTP Live Streaming** (**HLS**) and **Dynamic Adaptive Streaming over HTTP** (**DASH**), allows videos to be delivered in varying qualities, depending on network conditions. HLS is widely used for live and on-demand streaming on the web, as well as dynamically adjusting video quality based on the viewer's internet speed. Similarly, DASH is a flexible standard that enables high-quality streaming of media content over the internet.
- **Local and network videos**: It's suitable for playing videos stored locally on the device or streamed over the network.
- **Embedded video content**: VideoView is great for applications that need to embed video content directly within their UI, such as tutorial apps, video players, or social media apps with video feeds.

Now that we know its features and recommended use cases, let's look at an example so that we understand how it works. In this example, we're

using the 1.7.0 version of the **`androidx.media:media`** library:

```
@Composable
fun VideoPlayer(modifier: Modifier = Modifier, videoUrl:
String) {
    val context = LocalContext.current
    AndroidView(
        modifier = modifier,
        factory = { ctx ->
            VideoView(ctx).apply {
                val mediaController = MediaController(ctx)
                setMediaController(mediaController)
                mediaController.setAnchorView(this)
                setVideoURI(Uri.parse(videoUrl))
                start() // Auto-start playback
            }
        }
    )
}
```

Here, we start by declaring a composable called **`VideoPlayer`**. This composable accepts a **`videoUrl`** string as a parameter. This specifies the location of the video to be played.

Within the function, **`LocalContext.current`** is used to obtain the current context from the Compose environment. The **`AndroidView`** composable is then employed to bridge the gap between traditional Android UI components and the Compose world. It takes a factory Lambda expression where **`VideoView`** is instantiated by using the context.

Next, **`MediaController`** is created and associated with **`VideoView`** through **`setMediaController()`**, providing standard media controls such as play, pause, and seek to enhance user interaction with the video playback.

The media controller is anchored to **`VideoView`** using **`setAnchorView(this)`**, ensuring that the control interface is displayed correctly concerning the video view. The video URL that's passed to the function is parsed into a **`Uri`** component and set on **`VideoView`** with **`setVideoURI()`**, pointing the player to the video content.

Finally, **`start()`** is called on **`VideoView`** to initiate video playback automatically as soon as the setup is complete and the video is ready to be shown.

In this section, we took a sneak peek at how the MediaPlayer API and VideoView work and their features. Now, it's time for the crown jewel: ExoPlayer.

Understanding the basics of ExoPlayer

ExoPlayer stands as a significant advancement over Android's basic MediaPlayer, offering a level of flexibility, customization, and support for advanced streaming formats that MediaPlayer simply cannot match. This

superiority makes ExoPlayer the go-to choice for developers needing robust, feature-rich media playback capabilities in their applications.

One of ExoPlayer's most compelling advantages is its adaptability. Unlike the relatively static MediaPlayer, ExoPlayer can be easily adapted and extended to suit specific application needs. Its modular architecture allows developers to include only the components they need, reducing the app's overall size. Furthermore, ExoPlayer's customization options extend to its user interface, with the ability to create custom controls and layouts that seamlessly integrate with the rest of the application's design. This adaptability ensures that developers can craft a unique media playback experience that aligns perfectly with their app's branding and user interface guidelines.

In the realm of streaming, ExoPlayer's strengths become even more apparent. It offers out-of-the-box support for modern streaming protocols such as HLS and DASH. These adaptive streaming protocols are essential for delivering content efficiently over the internet, adjusting the quality of the stream in real time based on the user's current network conditions. This ensures an optimal viewing experience that minimizes buffering and playback interruptions even under fluctuating network speeds.

MediaPlayer, by contrast, offers limited support for such streaming protocols, often requiring developers to implement additional solutions or workarounds to achieve similar functionality. With ExoPlayer, developers gain direct access to these advanced features, simplifying the development process and enhancing the end user experience.

As we can see, ExoPlayer's functionality is widely superior due to its flexibility and wide format support and those are the reasons we will use it in this project. On the other hand, as it is more complex, we will have to learn more about it before we start to implement our video player using it.

Well, let's do exactly that and break down ExoPlayer's architecture.

Exploring ExoPlayer's architecture

ExoPlayer's architecture is designed to be both flexible and extensible, making it capable of handling a wide range of media playback scenarios. ExoPlayer has several core components that work together to provide a robust and efficient media playback experience. Understanding these components is key to leveraging ExoPlayer's full capabilities in our applications. Let's take a look at them here.

The ExoPlayer instance – the central media playback engine

The ExoPlayer instance itself acts as the central hub for media playback, orchestrating the interaction between the various components involved in the playback process, managing the playback state, and coordinating the fetching, decoding, and rendering of media. Unlike Android's MediaPlayer, which operates as a black box, ExoPlayer provides develop-

ers with detailed control over playback and access to the playback pipeline, enabling fine-tuned adjustments to fit the application's specific needs.

Here's a simple example of how to initialize ExoPlayer and prepare it to play a media item:

```
val context = ... // Your context here
val player = ExoPlayer.Builder(context).build().apply {
    // Media item to be played
    val mediaItem =
        MediaItem.fromUri("http://example.com/media.mp3")
    // Set the media item to be played
    setMediaItem(mediaItem)
    // Prepare the player
    prepare()
    // Start playback
    playWhenReady = true
}
```

The process begins with creating an **`ExoPlayer`** instance, utilizing a context-aware builder pattern that ensures the player is configured for the environment where it operates. Following its instantiation, a media item is specified through a URI, which could either point to a local resource or a remote media file. This media item is then associated with the **`ExoPlayer`** instance, indicating what content it should be prepared to play.

Once the media item has been set, the player enters a preparation phase by invoking the **`prepare()`** method. During this phase, ExoPlayer analyzes the media, setting up necessary buffers and decoding resources to ensure smooth playback.

The final step in the process involves setting the player's **`playWhenReady`** property to **`true`**, a command that triggers playback as soon as the player is fully prepared. This property provides flexibility, allowing developers to control when playback should start. This can be immediately after preparation or delayed based on additional conditions or user interactions.

MediaItem – sourcing the media resource

In ExoPlayer, **MediaItem** encapsulates details about a media source, such as its URI, metadata, and any configuration related to playback. It is a versatile and essential component that tells ExoPlayer what content to load and play. These are the key functions of MediaItem:

- **Media source specification**: The primary function of MediaItem is to specify the location of the media to be played. This can be a file path, a URL, or a content URI, among other formats.
- **Media configuration**: Beyond just specifying a media source, MediaItem allows for detailed configuration of the playback. This includes setting DRM configurations, specifying subtitles, and defining custom attributes through metadata.

- **Adaptive streaming**: For adaptive streaming content (such as DASH and HLS), MediaItem can include the necessary information for ExoPlayer to adapt the stream's quality dynamically based on network conditions. This information includes metadata such as the URLs of the various stream segments, available quality levels, and codecs.
- **Playback options**: Developers can use MediaItem to configure specific playback options, such as start and end positions, looping, and more. These options provide fine-grained control over how the media is played.

In practice, once a MediaItem is created and configured, it is passed to the ExoPlayer instance so that it can be prepared for playback. You can load a single MediaItem for simple playback scenarios or manage a playlist by loading multiple MediaItems. Let's see a brief example:

```
val mediaItem =
    MediaItem.fromUri("https://example.com/video.mp4")
player.setMediaItem(mediaItem)
player.prepare() // Prepares the player with the provided
                    MediaItem
player.playWhenReady = true // Starts playback as soon as
                               preparation is complete
```

In this example, we are creating **`mediaItem`** from a URL and preparing it to be reproduced by the ExoPlayer instance.

TrackSelector – managing media tracks

The **`TrackSelector`** instance is a critical component of ExoPlayer that's responsible for selecting the specific tracks to be played. A video might contain multiple audio tracks in different languages, several video qualities, or various subtitle tracks, and **`TrackSelector`** decides which of these tracks are best suited for the current playback context based on the device's capabilities, user preferences, and network conditions. This selection process is crucial for adaptive streaming scenarios as a single video is encoded at multiple quality levels and stored on the server.

Here's an example of its use:

```
val trackSelector =
    DefaultTrackSelector(context).apply {
        setParameters(buildUponParameters()
            .setPreferredAudioLanguage("en")
}
val player = ExoPlayer.Builder(context)
    .setTrackSelector(trackSelector)
    .build()
```

The process starts with the creation of a **`DefaultTrackSelector`** instance. The **`DefaultTrackSelector`** instance is a component of **`ExoPlayer`** that decides which tracks (audio, video, or text) are played from the media based on various criteria, such as the user's device capabilities and the tracks' properties. In this example, the track selector is configured to prefer au-

dio tracks in English. This preference is set by modifying the track selector's parameters, indicating that if the media contains multiple audio tracks in different languages, the English one should be chosen, if available.

After configuring the track selector, it's used in the construction of the **`ExoPlayer`** instance. Here, **`ExoPlayer.Builder`** is provided with the application context and the customized track selector when building the player. This ensures that when the **`ExoPlayer`** instance prepares and plays media, it uses the logic defined in **`DefaultTrackSelector`** for track selection. Essentially, this setup allows for more control over which audio track is selected during playback, based on the predefined criteria (in this case, the language preference).

This approach to configuring ExoPlayer is particularly beneficial in applications that deal with media containing multiple tracks for different audience demographics or in scenarios where the application needs to adhere to user preferences or settings, such as a language selection option. By customizing the track selector, developers can ensure that the media playback experience is optimized for the specific needs and preferences of their users, enhancing overall usability and satisfaction.

LoadControl – handling buffering and loading

The **`LoadControl`** component oversees the strategy for buffering and loading media resources. Efficient buffering is essential for smooth playback, especially in streaming scenarios where network conditions can vary widely. The **`LoadControl`** component determines how much media data to buffer at any given time, striking a balance between reducing initial loading times and minimizing the likelihood of playback interruptions. We can customize the buffering policy to cater to specific requirements, such as prioritizing quick start times or ensuring uninterrupted playback.

The following is an example of creating a custom **`LoadControl`** component to modify the buffer policy:

```
val loadControl = DefaultLoadControl.Builder().apply {
    // Set minimum buffer duration to 2 minutes
    setBufferDurationsMs(
        minBufferMs = 2 * 60 * 1000,
        maxBufferMs =
            DefaultLoadControl.DEFAULT_MAX_BUFFER_MS,
        bufferForPlaybackMs =
            DefaultLoadControl
            .DEFAULT_BUFFER_FOR_PLAYBACK_MS,
        bufferForPlaybackAfterRebufferMs =
            DefaultLoadControl
            .DEFAULT_BUFFER_FOR_PLAYBACK_AFTER_REBUFFER_MS
    )
}.build()
val player = ExoPlayer.Builder(context)
    .setLoadControl(loadControl)
    .build()
// Continue setting up the player as before
```

The example begins by creating an instance of `DefaultLoadControl` using `Builder`. Here, `DefaultLoadControl` is an implementation of the `LoadControl` interface provided by ExoPlayer and is designed to manage media buffering based on various parameters that I will explain now.

The `setBufferDurationsMs` method is called on the builder to specify custom buffer durations. Specifically, it sets the minimum buffer duration (`minBufferMs`) to 2 minutes (120,000 milliseconds). This means that ExoPlayer will attempt to buffer at least 2 minutes of media before starting playback, which can help ensure smooth playback under varying network conditions.

The other parameters (`maxBufferMs`, `bufferForPlaybackMs`, and `bufferForPlaybackAfterRebufferMs`) are set to their default values, which are predefined in `DefaultLoadControl`. These parameters control the maximum buffer size, the minimum amount of media that must be buffered for playback to start, and the minimum amount of media that must be buffered to resume playback after a rebuffer, respectively.

NOTE

If you want to learn more about the aforementioned options, you can find all the details in the documentation for `DefaultLoadControl.Builder`*:* **https://developer.android.com/reference/androidx/media3/exoplayer/DefaultLoadControl.Builder**.

After configuring the buffer durations, the `build()` method is called to create the `DefaultLoadControl` instance with the specified settings.

This custom `LoadControl` component is then set on a new `ExoPlayer` instance through the `setLoadControl` method of `ExoPlayer.Builder`. This step integrates the custom buffering strategy with the player, meaning that the player will use the specified buffer durations during playback.

Finally, the `build` method is called on `ExoPlayer.Builder` to create the `ExoPlayer` instance that was configured with the custom `LoadControl` component.

Renderers – rendering media to outputs

Renderers are the components that output the media to the appropriate destination, such as rendering video frames to the screen or audio samples to speakers. ExoPlayer uses separate renderers for different types of tracks, allowing for parallel processing and rendering of audio, video, and text tracks. This separation enables ExoPlayer to support a wide range of media types and formats efficiently. Moreover, developers can implement custom renderers to handle non-standard media types or apply special processing to media before playback.

To illustrate this, consider the following example, where a custom renderer is being used to apply a grayscale filter to video content:

```
class GrayscaleVideoRenderer(
    eventHandler: Handler,
    videoListener: VideoRendererEventListener,
    maxDroppedFrameCountToNotify: Int
) : SimpleDecoderVideoRenderer(eventHandler, videoListener, maxDroppedFrameCountToNotify) {
    override fun onOutputFormatChanged(format: Format,
    outputMediaFormat: MediaFormat?) {
        super.onOutputFormatChanged(format,
            outputMediaFormat)
        // Setup to modify the color format to grayscale
    }
    override fun renderOutputBufferToSurface(buffer:
    OutputBuffer, surface: Surface,
    presentationTimeUs: Long) {
        // Apply grayscale effect to the buffer before
           rendering to the surface
    }
}
```

The **`GrayscaleVideoRenderer`** class extends ExoPlayer's **`SimpleDecoderVideoRenderer`** to apply a grayscale effect to video frames during playback. This customization allows it to not just decode and display the video but also transform each frame to grayscale in real time, enhancing the visual presentation for stylistic choices or accessibility.

When initializing, this renderer takes a **`Handler`** component for thread-safe event dispatching, a **`VideoRendererEventListener`** component for managing video events, and an integer that sets the threshold for notifying about dropped frames. This setup helps keep the playback smooth and responsive.

It overrides the **`onOutputFormatChanged`** method, where it prepares for video format changes. This is where adjustments for grayscale processing would be set up. The **`renderOutputBufferToSurface`** method is where the grayscale effect is applied to each video frame before they are rendered to the screen.

Now that we are familiar with the most important components of ExoPlayer, let's integrate it into our project.

Integrating ExoPlayer into our project

To integrate ExoPlayer, we have to include the necessary library dependencies in our version catalog:

```
[versions]
...
exoPlayer = "1.2.1"
[libraries]
...
exoPlayer-core = { module = "androidx.media3:media3-exoplayer", version.ref = "exoPlayer" }
exoPlayer-ui = { module = " androidx.media3:media3-ui", version.ref = "exoPlayer" }
```

As with every dependency that we have included, we must add them to the **`build.gradle`** file of the module where we are going to use them. In

this case, we'll add it them the **`build.gradle`** file for **`:feature:playback`**:

```
dependencies {
    implementation(libs.exoPlayer.core)
    implementation(libs.exoPlayer.ui)
}
```

With these two dependencies, we have all the components we need to use ExoPlayer:

- **`androidx.media3:media3-exoplayer`**: This is the core module of ExoPlayer in the Media3 library. It includes the essential classes and interfaces needed for media playback functionality. This module provides the fundamental components for media playback, including the ExoPlayer interface, media source handling, and playback control logic. It is the backbone of media playback in Media3, offering high-performance, low-level media playback capabilities.
- **`androidx.media3:media3-ui`**: This module provides user interface components for media playback in the Media3 library. It includes pre-built UI components such as **`PlayerView`** (a view that displays video content and playback controls) and other UI elements for controlling media playback. These components can be customized or replaced with custom implementations if needed. This module helps developers quickly integrate ExoPlayer with a functional UI for media playback.

Now, we're all set. In the next section, we will build our playback UI and connect it with ExoPlayer.

Creating the video playback user interface

In this section, we're going to build the video playback UI and focus on the essentials: a title bar, close, play/pause, forward and rewind buttons, a progress bar, and a time indicator. We will start by creating the **`PlaybackScreen`** composable, the main composable for this new screen, after which we will add the additional components required to make it function.

Building PlaybackScreen and its composables

Let's start building the **`PlaybackScreen`** composable:

```
@Composable
fun PlaybackScreen() {
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Black)
    ) {
        TopMediaRow(Modifier.align(Alignment.TopCenter))
```

```
        PlayPauseButton(Modifier.align(Alignment.Center))
        ProgressBarWithTime(Modifier
            .align(Alignment.BottomCenter))
    }
}
```

We start by declaring a **`Box`** container that fills the entire screen and sets its background to black, mimicking the dark mode typically preferred in video playback interfaces. Within this **`Box`**, we place three key components that constitute our playback UI: a top media row, a play/pause button, and a progress bar with a time indicator.

Here, **`TopMediaRow`** is positioned at the top center of the screen, likely containing the title bar and close button. Then, **`PlayPauseButton`** is placed right in the center of the screen, making it easy for users to start or pause playback with a simple tap. Finally, **`ProgressBarWithTime`** is aligned at the bottom center, allowing users to see how much of the video has played and how much is left. Each of these components is aligned within the **`Box`** container using the **`Modifier.align`** method, ensuring they are positioned exactly where we want them in the UI.

Now that we have built the base of the screen, including every composable needed, it's time to build them. We will start with the **`TopMediaRow`** composable:

```
@Composable
fun TopMediaRow(modifier: Modifier = Modifier) {
    Row(
        modifier = modifier.fillMaxWidth().padding(20.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = "S1:E1 - Pilot", color = Color.White)
        Icon(imageVector = Icons.Default.Close,
            contentDescription = "Close",
                tint = Color.White)
    }
}
```

In this **`TopMediaRow`** composable function, we're designing the top part of our video playback UI, which is specifically tailored for displaying the episode information and a close button. This function uses a **`Row`** layout to arrange its elements horizontally across the screen. The modifier that's applied to this **`Row`** layout ensures it stretches to fill the maximum width of its parent container and applies a padding of 20 **density-independent pixels** (**dp**) around its edges for a neat, uncluttered look.

Within the **`Row`** layout, we use two main components: **`Text`** and **`Icon`**:

- The **`Text`** component displays the episode information, such as **S1:E1 'Pilot'**, in white color, making it easily visible against the dark background typical of video playback screens.
- The **`Icon`** component uses the default "close" symbol with its tint also set to white to maintain consistency and visibility. The **`horizontalArrangement`** property is set to **`Arrangement.SpaceBetween`** to ensure the

text and icon are placed on opposite ends of the row, while **`verticalAlignment`** keeps them centered vertically within the row.

Now, let's move to the next row, which contains the **`PlayPauseButton`** composable:

```
@Composable
fun PlayPauseButton(modifier: Modifier = Modifier) {
    Row(
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically,
        modifier = modifier
    ) {
        IconButton(
            modifier = Modifier.padding(20.dp),
            onClick = { /* Rewind action */ })
        {
            Icon(
                modifier = Modifier
                    .height(80.dp)
                    .width(80.dp),
                imageVector = Icons.Default.ArrowBack,
                contentDescription = "Rewind 10s",
                tint = Color.White)
        }
        IconButton(
            modifier = Modifier
                .padding(20.dp),
            onClick = { /* Play/Pause action */ }
        ) {
            Icon(
            modifier = Modifier
                .height(80.dp)
                .width(80.dp),
            imageVector = Icons.Default.PlayArrow,
            contentDescription = "Play/Pause",
            tint = Color.White)
        }
        IconButton(
            modifier = Modifier
                .padding(20.dp),
            onClick = { /* Fast-forward action */ }) {
            Icon(
                modifier = Modifier
                    .height(80.dp)
                    .width(80.dp),
                imageVector = Icons.Default.ArrowForward,
                contentDescription = "Fast-forward 10s",
                tint = Color.White)
        }
    }
}
```

The **`PlayPauseButton`** composable function will provide the central control mechanism for video playback and incorporate rewind, play/pause, and fast-forward actions within a single, intuitive interface. This function employs a **`Row`** layout to horizontally align its child elements – the buttons

for each control action – so that they're centered both horizontally and vertically.

Each button is created using the **`IconButton`** component. These buttons are spaced out with a padding of 20 dp to ensure they're comfortably tappable without the risk of accidental presses. The icons for rewind, play/pause, and fast-forward are sized uniformly at 80 dp by 80 dp, making them large enough to be easily tapped and visually recognized.

The **`Icon`** components within each **`IconButton`** are specifically chosen to visually represent their respective actions: an arrow pointing backward for rewind, a play arrow for play/pause, and an arrow pointing forward for fast-forward, each accompanied by a content description for accessibility purposes. The placeholder comments within the **`onClick`** parameters indicate where the functionality for each button – rewinding the video by 10 seconds, toggling between playing and pausing, and fast-forwarding by 10 seconds – would be implemented.

Finally, we have one last composable to build, the **`ProgressBarWithTime`** composable:

```
@Composable
fun ProgressBarWithTime(modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
            val progress = remember { mutableStateOf(0.3f)
                } // Dummy progress
            val formattedTime = "22:49" // Dummy time
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment =
                    Alignment.CenterVertically
            ) {
                Slider(
                    value = progress.value,
                    onValueChange =
                        { progress.value = it },
                    modifier = Modifier.weight(1f)
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(text = formattedTime,
                    color = Color.White)
            }
    }
}
```

This composable is wrapped in a **`Row`** layout, which spans the maximum width available (to accommodate the length of the video) and adjusts its height to wrap the content closely, ensuring a tidy appearance with ample padding around its edges for a balanced layout.

The core functionality centers around two elements:

- The **`Slider`** component represents the video's progress. It uses a mutable state initialized at **`0.3`** (30% progress) to simulate the current position of the video playback. This state is interactively adjustable, allowing users to seek through the video. The **`onValueChange`** event updates the progress state, reflecting the user's input. To visually separate the progress bar from the time indicator and to ensure the layout remains intuitive, a spacer is inserted between these elements, maintaining a clear distinction.
- Adjacent to the **`Slider`** component, the **`Text`** component displays the current playback time (set to **22:49** for now, until we integrate the playback functionality) in white color. The time is displayed to provide users with exact information about how much of the video has been played or how much is left, enhancing the user experience by offering precise control over video playback.

Although it may seem that our playback UI is complete, there is still one thing that we should take care of before integrating the playback feature itself. When we are watching a video, we don't want all those controls to be occupying the screen, making it difficult to watch the content. The controls usually disappear automatically after the user hasn't been interacting with the screen. So, let's implement this change.

Making the controls disappear when playing the content

We know our playback controls should disappear if they haven't been used for a while. The easiest way to do this is to have a value that will indicate if the controls should be visible or not, and we will modify its value to **`false`** when the screen has been idle for a time. Let's make these modifications in the **`PlaybackScreen`** composable, as follows:

```
@Composable
fun PlaybackScreen() {
    val isControlsVisible = remember { mutableStateOf(true) }
    val coroutineScope = rememberCoroutineScope()
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Black)
            .pointerInput(Unit) {
                detectTapGestures(
                    onPress = {
                        // Reset the visibility timer on
                           user interaction
                        isControlsVisible.value = true
                        coroutineScope.launch {
                            delay(15000) // 15 seconds
                                         delay
                            isControlsVisible.value = false
                        }
                    }
                )
```

```
            }
        ) {
            if (isControlsVisible.value) {
                TopMediaRow(Modifier.align(Alignment.TopCenter))
                PlayPauseButton(Modifier.align(
                    Alignment.Center))
                ProgressBarWithTime(Modifier.align(
                    Alignment.BottomCenter))
            }
        }
    }
```

The core idea of these modifications is to track user interaction and use a timer to determine when to hide the controls. Initially, as mentioned previously, we'll introduce a state to manage the visibility of the controls. This state will likely be a Boolean that toggles between visible and invisible (**`true`** and **`false`**) based on user interaction and the passage of time without interaction.

For detecting user interactions, we could wrap the **`Box`** layout that contains our playback UI components in a **`Modifier.pointerInput`** Lambda. Inside this Lambda, we can listen for touch input events, and each time a touch is detected, we can reset the timer – a coroutine launched with **`LaunchedEffect`** keyed to the visibility state might handle this. This coroutine will wait for 15 seconds of inactivity (no touch events detected) before setting the controls' visibility state to **`false`**, effectively hiding them. To ensure the controls reappear when the user interacts with the screen again, the same touch input detection mechanism will set the visibility state back to **`true`**, and the coroutine will restart its countdown.

Incorporating this functionality requires making modifications to the **`PlaybackScreen`** composable function so that it includes state handling for visibility and can modify the **`TopMediaRow`**, **`PlayPauseButton`**, and **`ProgressBarWithTime`** functions so that they accept and react to the visibility state. This means each of these components will only be rendered when the state indicates they should be visible.

Once we've finished, our playback UI should look like this:

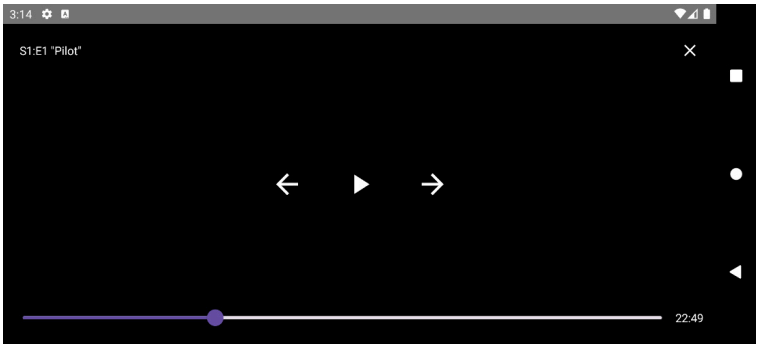

Figure 8.1: Finished playback UI (with controls shown)

When the controls are hidden, it should just show the video content (this isn't visible yet as it hasn't been implemented):

Figure 8.2: Finished playback UI (with controls hidden)

In this section, we created a UI to display the videos. In the next section, we will integrate ExoPlayer so that our app can start playing videos.

Playing video using ExoPlayer

In this section, we'll harness the full power of ExoPlayer so that we can integrate it into our newly created video playback UI. Let's learn how we can do this.

Creating PlaybackActivity

We'll start by creating a new **`Activity`** for this functionality called **`PlaybackActivity`**:

```
class PlaybackActivity: ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PlaybackScreen()
        }
    }
}
```

This **`PlaybackActivity`** activity will show our already created **`PlaybackScreen()`** in its content.

We also want our playback UI to be always displayed in landscape mode. To do so, we'll configure this activity in the **`AndroidManifest.xml`** file, as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android =
"http://schemas.android.com/apk/res/android">
    <application>
        <activity android:name =
        "com.packt.playback.presentation.PlaybackActivity"
        android:screenOrientation="landscape"/>
    </application>
</manifest>
```

Here, we are declaring **`PlaybackActivity`** so that it has landscape as a forced screen orientation. This will ensure it will only be rendered in

landscape mode, despite what orientation the user is holding their phone.

Creating PlaybackViewModel

Now, we need to create the player, which is the component that's responsible for managing the media playback. We will create **`PlaybackViewModel`** to handle the ExoPlayer instance and all the logic needed for the view to interact with the video player and watch the media.

To start, we are going to build the basic setup logic for our player in **`PlaybackViewModel`**:

```
@HiltViewModel
class PlaybackViewModel @Inject constructor(): ViewModel()
{
    lateinit var player: ExoPlayer
    @OptIn(UnstableApi::class)
    private fun preparePlayerWithMediaSource(exoPlayer:
    ExoPlayer) {
        val mediaUrl = "https://example.com/media.mp4"
        val mediaSource = ProgressiveMediaSource.Factory(
            DefaultHttpDataSource.Factory())
            .createMediaSource(MediaItem.fromUri(mediaUrl))
        exoPlayer.setMediaSource(mediaSource)
        exoPlayer.prepare()
    }
    fun setupPlayer(context: Context) {
        player = ExoPlayer.Builder(context).build().also {
        exoPlayer ->
            preparePlayerWithMediaSource(exoPlayer)
        }
    }
    override fun onCleared() {
        super.onCleared()
        player.release()
        progressUpdateJob?.cancel()
    }
}
```

This is the start of our **`PlaybackViewModel`** composable, which is designed to manage the media playback functionality of an Android app.

The core component of this **`ViewModel`** is the ExoPlayer instance, which is stored in a property named **`player`**. This **`player`** property is responsible for all media playback operations. However, when **`ViewModel`** is first created, the player is not initialized; it's declared with **`lateinit`**, meaning it will be initialized later but before any other component needs to access it.

The **`setupPlayer`** function is publicly exposed and intended to be called with a **`Context`** object, which provides access to application-specific resources and classes. Inside this function, **`ExoPlayer.Builder`** is used to create an instance of **`ExoPlayer`**. This setup process involves calling the **`build()`** method on the builder, which returns a fully configured **`ExoPlayer`** instance. Immediately after creating this instance, the **`also`**

block executes, calling the **`preparePlayerWithMediaSource`** method with the newly created player.

The **`preparePlayerWithMediaSource`** method is where the actual media source is set up. It takes an **`ExoPlayer`** instance as an argument and configures it to play a specific media file. The URL of the media file is defined as **https://example.com/media.mp4**. To play this media, **`ProgressiveMediaSource`** is created, which is suitable for playing regular media files such as MP4s over HTTP. This media source is then attached to the **`ExoPlayer`** instance using the **`setMediaSource`** method, and **`prepare()`** is called to prepare the player for playback. It's worth noting that this method is marked as private, meaning it's intended to be used only within the **`PlaybackViewModel`** class. The **`@OptIn(UnstableApi::class)`** annotation indicates that this method uses APIs that are not yet stable and may change in the future.

Lastly, the **`onCleared`** method overrides a **`ViewModel`** life cycle callback that gets called when **`ViewModel`** is about to be destroyed. This method releases the **`ExoPlayer`** instance by calling **`player.release()`**, ensuring that resources are freed and preventing memory leaks.

Now, we'll add the view that will render the media content in **`PlaybackScreen`** and connect it to the player:

```
@Composable
fun PlaybackScreen() {
    val viewModel: PlaybackViewModel = hiltViewModel()
    val isControlsVisible = remember { mutableStateOf(true) }
    val coroutineScope = rememberCoroutineScope()
    viewModel.setupPlayer(LocalContext.current)
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Black)
            .pointerInput(Unit) {
                detectTapGestures(
                    onPress = {
                        isControlsVisible.value = true
                        coroutineScope.launch {
                            delay(15000) // 15 seconds
                                         delay
                            isControlsVisible.value = false
                        }
                    }
                )
            }
    ) {
        VideoPlayerComposable(
            modifier = Modifier.matchParentSize(),
            player = viewModel.player
        )
        if (isControlsVisible.value) {
            ...
        }
```

```
    }
}
```

In the **`PlaybackScreen`** composable, we obtain an instance of **`PlaybackViewModel`** using **`hiltViewModel()`**. This **`ViewModel`** is central to managing the media playback life cycle and interactions within the app.

Once **`ViewModel`** is ready, we call **`viewModel.setupPlayer(LocalContext.current)`** to initialize **`ExoPlayer`**. This setup is crucial because it prepares the player with the appropriate Android context, allowing it to load and play media files effectively. Ensuring that ExoPlayer is initialized with the current context helps manage resources efficiently, which is essential for smooth playback.

The UI component responsible for displaying the video is **`VideoPlayerComposable`**. We pass the initialized player from **`ViewModel`** to this composable, which is placed inside a **`Box`** layout. This layout is configured to fill the maximum size of its parent and sets a black background to emphasize the video content. The **`Box`** layout also handles user interactions, listening for tap gestures to toggle the visibility of playback controls. When a tap is detected, it makes the controls visible and starts a coroutine that hides these controls again after 15 seconds if no further interaction occurs.

Inside the **`Box`** layout, conditional logic checks the value of **`isControlsVisible`**. If **`true`**, playback controls are rendered on top of the video. This allows users to interact with the video, such as pausing, skipping, or adjusting the volume, but only when they choose to display the controls.

Finally, we will explore how to implement **`VideoPlayerComposable`** so that we can effectively utilize the player to render the video while responding dynamically to user interactions with playback controls.

Let's see how we can implement this new composable. Unfortunately, at the time of writing, the library doesn't provide a Jetpack Compose option to show the player, so we need to create one inside an **`AndroidView`** composable, as follows:

```
@Composable
fun VideoPlayerComposable(
    modifier: Modifier = Modifier,
    player: ExoPlayer
) {
    AndroidView(
        factory = { ctx ->
            PlayerView(ctx).apply {
                layoutParams = ViewGroup.LayoutParams(
                    MATCH_PARENT, MATCH_PARENT)
                setPlayer(player)
                useController = false
            }
        },
        modifier = modifier,
        update = { view ->
            view.player = player
```

```
        }
    )
}
```

The **`VideoPlayerComposable`** function takes two parameters:

- The **`Modifier`** instance allows you to customize the layout or appearance of this composable when it's used elsewhere in your UI
- The **`ExoPlayer`** instance is the media player that will handle the actual playback of the video content

Inside the **`AndroidView`** composable, the factory Lambda is where the traditional Android view is created – in this case, **`PlayerView`**. Here, **`PlayerView`** is a view provided by the **`ExoPlayer`** library to display video content and playback controls. Here, it's initialized with the application context (**`ctx`**).

After creating **`PlayerView`**, some properties are set on it:

- Here, **`layoutParams`** is set to **`MATCH_PARENT`** for both width and height, making **`PlayerView`** fill the entire space allocated to it. This ensures that the video will take up as much space as possible, typically the entire screen or the parent container.
- Then, **`setPlayer(player)`** attaches the passed **`ExoPlayer`** instance to **`PlayerView`**. This connection is what allows the video loaded in **`ExoPlayer`** to be displayed in this view.
- Finally, **`useController`** is set to **`false`**, indicating that the default playback controls provided by **`PlayerView`** (such as play, pause, and seek bar) will not be used. We will implement our own controls next.

Finally, the update Lambda of **`AndroidView`** is where you can update the properties of **`PlayerView`** based on changes to the composable's state or properties.

With these changes, our player is ready to start rendering the media via **`ViewPlayer`**. But we still have work to do. We need to bind the already developed controls to the player controls and keep the time and the progress bar of the video updated.

Connecting the controls with ExoPlayer

Let's start modifying the **`PlayPauseButton`** composable. In this case, we will need to bind the control functions with the ViewModel:

```
@Composable
fun PlayPauseButton(
    isPlaying: Boolean,
    onRewind: () -> Unit,
    onPlayPause: () -> Unit,
    onFastForward: () -> Unit,
    modifier: Modifier = Modifier,
    ) {
    Row(
        horizontalArrangement = Arrangement.Center,
```

```
            verticalAlignment = Alignment.CenterVertically,
            modifier = modifier
        ) {
            IconButton(
                onClick = onRewind,
                modifier = Modifier.padding(20.dp)
            ) {
                Icon(
                    modifier = Modifier
                        .height(80.dp)
                        .width(80.dp),
                    imageVector = Icons.Default.ArrowBack,
                    contentDescription = "Rewind 10s",
                    tint = Color.White
                )
            }
            IconButton(
                onClick = onPlayPause,
                modifier = Modifier.padding(20.dp)
            ) {
                Icon(
                    modifier = Modifier
                        .height(80.dp)
                        .width(80.dp),
                    imageVector = if (isPlaying)
                        Icons.Default.Close else
                        Icons.Default.PlayArrow,
                    contentDescription = if (isPlaying) "Pause"
                        else "Play",
                    tint = Color.White
                )
            }
            IconButton(
                onClick = onFastForward,
                modifier = Modifier.padding(20.dp)
            ) {
                Icon(
                    modifier = Modifier
                        .height(80.dp)
                        .width(80.dp),
                    imageVector = Icons.Default.ArrowForward,
                    contentDescription = "Fast-forward 10s",
                    tint = Color.White
                )
            }
        }
    }
```

Now, the **`PlayPauseButton`** composable takes several parameters, each serving a specific purpose within the UI component:

- **`isPlaying`** (Boolean): This parameter indicates the current playback state of the video. It is used to determine which icon to display on the play/pause button – either a play icon when the video is paused or a pause icon when the video is actively playing. This allows for intuitive control interactions from the user's perspective.
- **`onRewind`** (Lambda function): This is a callback function that's triggered when the user presses the rewind button. It should contain the

logic for what happens when the video is rewound, such as moving the playback position backward by a fixed amount.

- **`onPlayPause`** (Lambda function): This function is executed when the play/pause button is pressed. It handles toggling between playing and pausing the video based on the current state, facilitating seamless user control over video playback.
- **`onFastForward`** (Lambda function): Similar to **`onRewind`**, this callback is activated when the fast-forward button is pressed. It controls the logic for fast-forwarding the video, advancing the playback position forward by a predetermined interval.
- **`modifier`** (modifier): This parameter allows the appearance and layout of the button row within the composable to be customized. As we've seen previously, wt can be used to apply padding, define alignment, and set dimensions.

Now that we've added these new parameters, we need to pass them from the parent composable. Here's how you can include and invoke this composable with the required parameters:

```
val isPlaying = viewModel.isPlaying.collectAsState()
PlayPauseButton(
    isPlaying = isPlaying.value,
    onRewind = { viewModel.rewind() },
    onFastForward = {viewModel.fastForward() },
    onPlayPause = {viewModel.togglePlayPause() },
    modifier = Modifier.align(Alignment.Center)
)
```

As we can see, we have bound every Lambda parameter to **`ViewModel`** functions (that are yet to be implemented) and we are providing an **`isPlaying`** state to reflect the current playing status of the player.

Now, let's implement those functions in **`ViewModel`**:

```
private val _isPlaying = MutableStateFlow<Boolean>(false)
val isPlaying: MutableStateFlow<Boolean> = _isPlaying
fun setupPlayer(context: Context) {
    player = ExoPlayer.Builder(context).build().also {
    exoPlayer ->
        preparePlayerWithMediaSource(exoPlayer)
        exoPlayer.addListener(object : Player.Listener {
            override fun onIsPlayingChanged(isPlaying:
            Boolean) {
                _isPlaying.value = isPlaying
            }
            override fun onPlaybackStateChanged(
            playbackState: Int) {
                super.onPlaybackStateChanged(playbackState)
            }
            override fun onPositionDiscontinuity(
            oldPosition: Player.PositionInfo, newPosition:
            Player.PositionInfo, reason: Int) {
                super.onPositionDiscontinuity(oldPosition,
                    newPosition, reason)
            }
```

```
            override fun onTimelineChanged(timeline:
            Timeline, reason: Int) {
                super.onTimelineChanged(timeline, reason)
            }
        })
    }
}
fun togglePlayPause() {
    if (player.isPlaying) {
        player.pause()
    } else {
        player.play()
    }
}
fun rewind() {
    val newPosition =
        (player.currentPosition - 10000).coerceAtLeast(0)
    player.seekTo(newPosition)
}
fun fastForward() {
    val newPosition =
        (player.currentPosition + 10000)
            .coerceAtMost(player.duration)
    player.seekTo(newPosition)
}
```

First, we have defined a private mutable state flow, **`_isPlaying`**, to track whether the video is currently playing. This same state flow is exposed as a public **`MutableStateFlow`** component named **`isPlaying`**. In this case, **`isPlaying`** acts as a single source of truth for the playback state, allowing our UI components to update reactively based on whether the video is playing or paused.

The **`setupPlayer`** function, which we've already implemented, initializes the **`ExoPlayer`** instance. Now, it also attaches a listener to respond to playback events. The listener added overrides several methods, but most importantly, **`onIsPlayingChanged`** is used to update **`_isPlaying.value`** based on the player's state.

We've also included the functions to manipulate playback that we were already being called from the composable:

- **`togglePlayPause`**: This checks if the player is currently playing and toggles between play and pause. This method directly controls the player's state, making it the primary way the user interacts with the playback.
- **`rewind`** and **`fastForward`**: These options calculate a new position based on the current playback position and seek to that position. The **`rewind`** function moves the playback position backward by 10 seconds, while **`fastForward`** moves it forward by 10 seconds. These methods enhance user control over the video, allowing for quick navigation within the content.

Now, let's connect the next (and last) composable, **`ProgressBarWithTime`**:

```
@Composable
fun ProgressBarWithTime(
    currentPosition: Long,
    duration: Long,
    onSeek: (Long) -> Unit,
    modifier: Modifier = Modifier,
) {
    val progress =
        if (duration > 0) currentPosition.toFloat() /
            duration else 0f
    val formattedTime =
        "${formatTime(currentPosition)} /
            ${formatTime(duration)}"
    Row(
        modifier = modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Slider(
            value = progress,
            onValueChange = { newValue ->
                val newPosition =
                    (newValue * duration).toLong()
                onSeek(newPosition)
            },
            modifier = Modifier.weight(1f),
            valueRange = 0f..1f
        )
        Spacer(modifier = Modifier.width(8.dp))
        Text(text = formattedTime, color = Color.White)
    }
}
```

The function now accepts three new parameters: **`currentPosition`** and **`duration`** to represent the current playback position and the total length of the video in milliseconds, respectively, and an **`onSeek`** Lambda function that defines what to do when the user seeks to a new position.

The **`progress`** variable calculates how far along the video is, represented as a float between 0 and 1. This is achieved by dividing **`currentPosition`** by **`duration`**, which specifies a proportion of the video that has been played. If the duration is 0 (to avoid division by zero), progress is set to **`0f`**, indicating no progress.

The **`formattedTime`** string provides a user-friendly display of the current position and total duration of the video by using a custom formatting function, **`formatTime()`** (as we'll see next), to convert milliseconds into a more readable format (HH:MM:SS).

Finally, slider progress is now bound to the progress value, and its **`onValueChange`** event is wired to call **`onSeek`** with the new position when the user interacts with it. This allows the user to seek through the video by

moving the slider, with the **`onSeek`** function updating the video playback position accordingly.

Regarding the aforementioned **`formatTime`** function, it will work as follows:

```
fun formatTime(millis: Long): String {
    val totalSeconds = millis / 1000
    val hours = totalSeconds / 3600
    val minutes = (totalSeconds % 3600) / 60
    val seconds = totalSeconds % 60
    return if (hours > 0) {
        String.format("%02d:%02d:%02d", hours, minutes,
            seconds)
    } else {
        String.format("%02d:%02d", minutes, seconds)
    }
}
```

The input to the function is **`millis`**, which represents the time duration in milliseconds. This is a common way to represent time in programming because it's precise. However, milliseconds aren't very human-friendly, so the first step inside the function is to convert milliseconds into total seconds by dividing by **`1000`**. We're doing this because there are 1,000 milliseconds in a second.

Once you have the total seconds, the function calculates hours, minutes, and seconds. It divides the total seconds by **`3600`** (the number of seconds in an hour) to get hours. The remainder from that division (using the modulo operator, **`%`**) is then used to calculate minutes by dividing by **`60`** (since there are 60 seconds in a minute). Finally, the remainder from the minutes calculation gives you the seconds.

The last part is where the function formats the time string. If the duration includes hours (that is, if the duration is longer than 60 minutes), it formats the time as HH:MM:SS using **`String.format()`**. This method is used to create a formatted string with placeholders (**`%02d`**) for hours, minutes, and seconds. Here's a breakdown of the format:

- The **`%`** symbol indicates the start of a format specifier.
- The **`0`** specifies that the number should be padded with leading zeros if it has fewer digits than specified.
- The **`2`** indicates that the number should be at least two digits long.
- The **`d`** stands for 'decimal' and specifies that the placeholder is for an integer number.

So, **`%02d`** ensures that the number is at least two digits long and padded with zeros if necessary.

Going back to the composable, we also need to modify where **`ProgressBarWithTime`** is called in **`PlaybackScreen`**:

```
val currentPosition =
```

```
    viewModel.currentPosition.collectAsState()
val duration = viewModel.duration.collectAsState()
ProgressBarWithTime(
    currentPosition = currentPosition.value,
    duration = duration.value,
    onSeek = { newPosition ->
        viewModel.seekTo(newPosition)
    },
    modifier = Modifier.align(Alignment.BottomCenter)
)
```

As we can see, we have bound the **`seekTo`** Lambda parameter to a **`ViewModel`** function (that is yet to be implemented) and we are also providing **`duration`** and **`currentPosition`** states.

Now, let's modify **`PlaybackViewModel`** so that we can implement the pending functions related to the progress bar.

Implementing the video controls in PlaybackViewModel

The last step to make the progress bar work is to modify **`PlaybackViewModel`**. We can add the necessary functionality to control the progress bar like so:

```
private val _currentPosition = MutableStateFlow<Long>(0L)
val currentPosition: StateFlow<Long> = _currentPosition
private val _duration = MutableStateFlow<Long>(0L)
val duration: MutableStateFlow<Long> = _duration
private var progressUpdateJob: Job? = null
fun setupPlayer(context: Context) {
    player = ExoPlayer.Builder(context).build().also {
    exoPlayer ->
        preparePlayerWithMediaSource(exoPlayer)
        exoPlayer.addListener(object : Player.Listener {
            override fun onIsPlayingChanged(isPlaying:
            Boolean) {
                _isPlaying.value = isPlaying
                if (isPlaying) {
                    startPeriodicProgressUpdate()
                } else {
                    progressUpdateJob?.cancel()
                }
            }
            override fun onPlaybackStateChanged
            (playbackState: Int) {
                super.onPlaybackStateChanged(playbackState)
                if (playbackState == Player.STATE_READY ||
                playbackState == Player.STATE_BUFFERING) {
                    _duration.value = exoPlayer.duration
                }
            }
            override fun onPositionDiscontinuity(
            oldPosition: Player.PositionInfo, newPosition:
            Player.PositionInfo, reason: Int) {
                super.onPositionDiscontinuity(oldPosition,
                    newPosition, reason)
                _currentPosition.value =
```

```
                    newPosition.positionMs
            }
            override fun onTimelineChanged(timeline:
            Timeline, reason: Int) {
                super.onTimelineChanged(timeline, reason)
                if (!timeline.isEmpty) {
                    _duration.value = exoPlayer.duration
                }
            }
        })
    }
}
private fun startPeriodicProgressUpdate() {
    progressUpdateJob?.cancel()
    progressUpdateJob = viewModelScope.launch {
        while (coroutineContext.isActive) {
            val currentPosition = player.currentPosition
            _currentPosition.value = currentPosition
            delay(1000)
        }
    }
}
fun seekTo(position: Long) {
    if (::player.isInitialized && position >= 0 &&
    position <= player.duration) {
        player.seekTo(position)
    }
}
override fun onCleared() {
    super.onCleared()
    player.release()
    progressUpdateJob?.cancel()
}
```

With that, we've declared private mutable state flows called **`_currentPosition`** and **`_duration`** for tracking the current playback position and the total video duration, respectively. These are exposed as read-only StateFlows to the rest of the app, ensuring that the UI components can observe these values and react to changes, but cannot modify them directly.

The listener in the **`setupPlayer`** function has also been modified to include functionality to keep the two states, **`_currentPosition`** and **`_duration`**. The following modifications have been made to the listener callbacks:

- **`onIsPlayingChanged`**: This updates the **`_isPlaying`** state and controls the start and stop of a job, which periodically updates the current playback position. This is essential for keeping the UI in sync with the actual playback.
- **`onPlaybackStateChanged`**: This checks if the player is ready or buffering and updates the **`_duration`** state with the total duration of the video. This is necessary for setting up the progress bar.
- **`onPositionDiscontinuity`** and **`onTimelineChanged`**: These ensure that changes in the video playback position or timeline (such as seeking or switching to another video) update **`_currentPosition`** and **`_duration`** correctly.

Then, the new function, **`startPeriodicProgressUpdate`**, launches a coroutine that periodically updates the **`_currentPosition`** state with the player's current position. This loop runs every second, providing a near-real-time update of the playback position to the UI. It's crucial for making the progress bar move smoothly as the video plays.

Building on this functionality, the **`seekTo`** function allows the video to be seeked to a new position. It checks that the position is within the bounds of the video before calling **`seekTo`** on the ExoPlayer instance, effectively letting the user jump to different parts of the video through the progress bar.

Finally, the **`onCleared`** method has been modified to cancel the new **`progressUpdateJob`** composable in case we have to release the resources.

With these changes, our video player is ready. We just have to modify the hardcoded media URL in **`PlaybackViewModel`** (**`val mediaUrl = "https://example.com/media.mp4"`**) so that we can provide a URL to an actual video and let the magic happen! At this point, we should see the playback of the provided video.

In the last section of this chapter, we are going to enhance the functionality of our video player a little bit further by learning how to add subtitles.

Adding subtitles to the video player

In this section, we'll be adding subtitles to our video player. Subtitles are crucial for making videos accessible to everyone, but they can also be great for watching videos in noisy environments or when you need to keep the volume down. In this section, we'll learn how to load and display subtitles alongside our video while handling various formats and ensuring they sync up perfectly with our content.

To add subtitles, follow these steps:

1. Create a MediaSource for your video file, just as you would for any video playback in ExoPlayer. We did this in the previous section.
2. Create a MediaSource for your subtitle file. This often involves using **`SingleSampleMediaSource`** for single subtitle files or similar approaches for different formats.
3. Use **`MergingMediaSource`** to combine the video and subtitle sources. This merged source is then passed to the ExoPlayer instance for playback.
4. Initialize ExoPlayer with the merged source; it will handle the playback of both video and subtitles.

ExoPlayer supports a wide range of subtitle formats so that it can cater to various use cases and standards. Some of the most popular formats are as follows:

- **WebVTT** (`.vtt`): A widely used format for HTML5 video subtitles that's supported by many web browsers and platforms:
  - **Advantages**: WebVTT is extensively supported across most modern web browsers, making it ideal for online streaming services. It offers options for styling, positioning, and cue settings, allowing for a customizable viewing experience.
  - **Disadvantages**: Compared to simpler formats such as SRT, WebVTT's additional features can make it more complex to create and edit. Also, different platforms and browsers may interpret styling and formatting cues differently, leading to inconsistent presentations.
- **SubRip** (`.srt`): One of the most common subtitle formats that's simple in structure and supported by a wide range of media players:
  - **Advantages**: The structure of SRT files is straightforward, making them easy to create, edit, and debug. It is also supported by almost all media players, making it universally applicable for offline and online video playback.
  - **Disadvantages**: It provides basic text formatting, which limits its ability to customize the appearance of subtitles.

To give you an idea of what one of these formats looks like, here's an example of the content of a SubRip (`.srt`) file:

```
1
00:00:01,000 --> 00:00:03,000
Hello, welcome to our video!
2
00:00:05,000 --> 00:00:08,000
Today, we'll be discussing how to create a simple SRT file.
3
00:00:10,000 --> 00:00:12,000
Let's get started.
4
00:00:15,000 --> 00:00:20,000
Subtitles primarily enhance accessibility and also can be very helpful for understanding dialogue,
5
00:00:22,500 --> 00:00:25,000
And that's all there is to it!
```

Each block starts with a sequence number (for example, 1, 2, 3, and so on), followed by the time range on the next line (start time --> end time), and then the text of the subtitle. This text can be one or more lines and is followed by a blank line to indicate the end of the subtitle entry. This format can be edited with any text editor and saved with the `.srt` extension.

Now that we know a bit more about how to add subtitles to a video in ExoPlayer, let's add them by default to our already-implemented playback functionality. We just need to change the logic for the player setup in `PlaybackViewModel`:

```
@OptIn(UnstableApi::class)
private fun preparePlayerWithMediaSource(exoPlayer:
ExoPlayer) {
```

```
        val mediaUrl = "https://example.com/media.mp4"
        val subtitleUrl =
            "https://example.com/subtitles.srt"
        val videoMediaSource =
            ProgressiveMediaSource.Factory(
                DefaultHttpDataSource.Factory()
        ).createMediaSource(MediaItem.fromUri(mediaUrl))
        val subtitleSource =
            MediaItem.SubtitleConfiguration.Builder(
                Uri.parse(subtitleUrl)).build()
        val subtitleMediaSource =
            SingleSampleMediaSource.Factory(
                DefaultHttpDataSource.Factory()
        ).createMediaSource(subtitleSource, C.TIME_UNSET)
        val mergedSource =
            MergingMediaSource(videoMediaSource,
                subtitleMediaSource)
        exoPlayer.setMediaSource(mergedSource)
        exoPlayer.prepare()
}
```

In the preceding code, we modified the already existing **`preparePlayerWithMediaSource`** function. We started by adding a new media with the subtitles URL.

Then we created **`MediaSource`** for the subtitles, and we created a **`MediaItem.SubtitleConfiguration`** object from the subtitle URL (**`subtitleUrl`**). This configuration specifies how the subtitle should be loaded and displayed.

Then, **`SingleSampleMediaSource`** is created for the subtitle configuration. Here, **`SingleSampleMediaSource`** is used because subtitle files are typically a single piece of content rather than streamed content. The **`createMediaSource`** method here is slightly different from the video one; it takes the subtitle configuration and a duration parameter, which is set to **`C.TIME_UNSET`** to indicate that the duration is unknown or should be determined from the content itself.

Once both the video and subtitle sources have been created, they're combined into a single source using **`MergingMediaSource`**. This merged source tells **`ExoPlayer`** to play the video with the subtitles overlaying it.

Finally, the merged source is set on the **`ExoPlayer`** instance with **`setMediaSource`**, and **`prepare()`** is called. This action causes **`ExoPlayer`** to load the media and get ready for playback. When the video plays, the subtitles from the specified SRT file will be displayed at the correct times, as defined in the file.

The following figure shows the subtitles added:

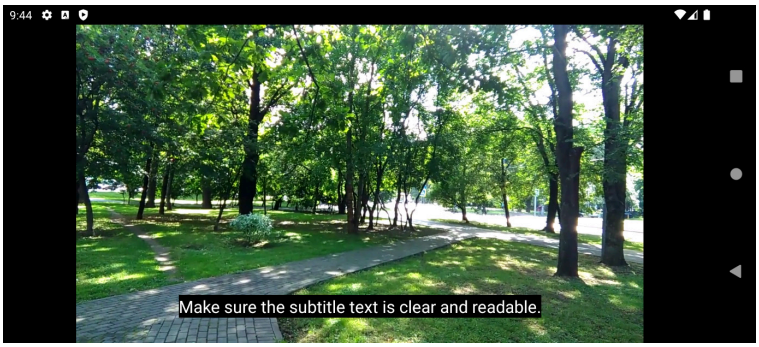

Figure 8.3: Playback with subtitles

With that, our player is ready to play back videos. By including subtitles, it offers a more accessible experience for our users.

Summary

In this chapter, we tackled the essentials of adding video playback in our Android app while focusing on the powerful ExoPlayer library. We started by comparing media options in Android before quickly realizing ExoPlayer's superiority due to its flexibility and wide format support. This set the stage for us to learn how ExoPlayer fits into an app and how to use it for playing videos smoothly.

We then walked through building a user-friendly video playback interface, covering everything from setting up ExoPlayer to managing playback controls. Finally, we explored adding subtitles to make your videos accessible to a wider audience, highlighting ExoPlayer's capability to enhance inclusivity.

Now that you have a solid grasp of video playback using ExoPlayer, you're ready to elevate your app with picture-in-picture mode and media casting.